



RELATÓRIO TÉCNICO: TELEMETRIA E AVALIAÇÃO

DE COMPONENTES IoT

- **ALUNO:** JOÃO PAULO NEGREIROS DO MONTE BORGES LEAL
- **ORIENTADOR:** PROF. DR. LUIZ OLÍMPIO
- **INSTITUIÇÃO:** UNIVERSIDADE FEDERAL DO CEARÁ (UFC) - CAMPUS RUSSAS

1. INTRODUÇÃO E OBJETIVOS

Este relatório técnico documenta as atividades de desenvolvimento, integração e testes realizadas no âmbito do projeto de telemetria e monitorização térmica híbrida. O principal objetivo do sistema é recolher, de forma síncrona e em tempo real, dados de temperatura e pressão atmosférica utilizando diferentes sensores físicos acoplados a uma única plataforma de processamento.

Adicionalmente, o projeto visa estabelecer uma infraestrutura de software robusta, garantindo a divisão clara entre a recolha física de dados (hardware), a organização e persistência (backend) e a análise visual (frontend).

2. AVALIAÇÃO DE HARDWARE: MÓDULOS ESP32

Foi realizado um procedimento de triagem técnica nas quatro placas de desenvolvimento ESP32 disponibilizadas no laboratório, com o objetivo de certificar a integridade física e operacional dos módulos para posterior devolução organizada.

2.1. Metodologia de Testes Aplicada

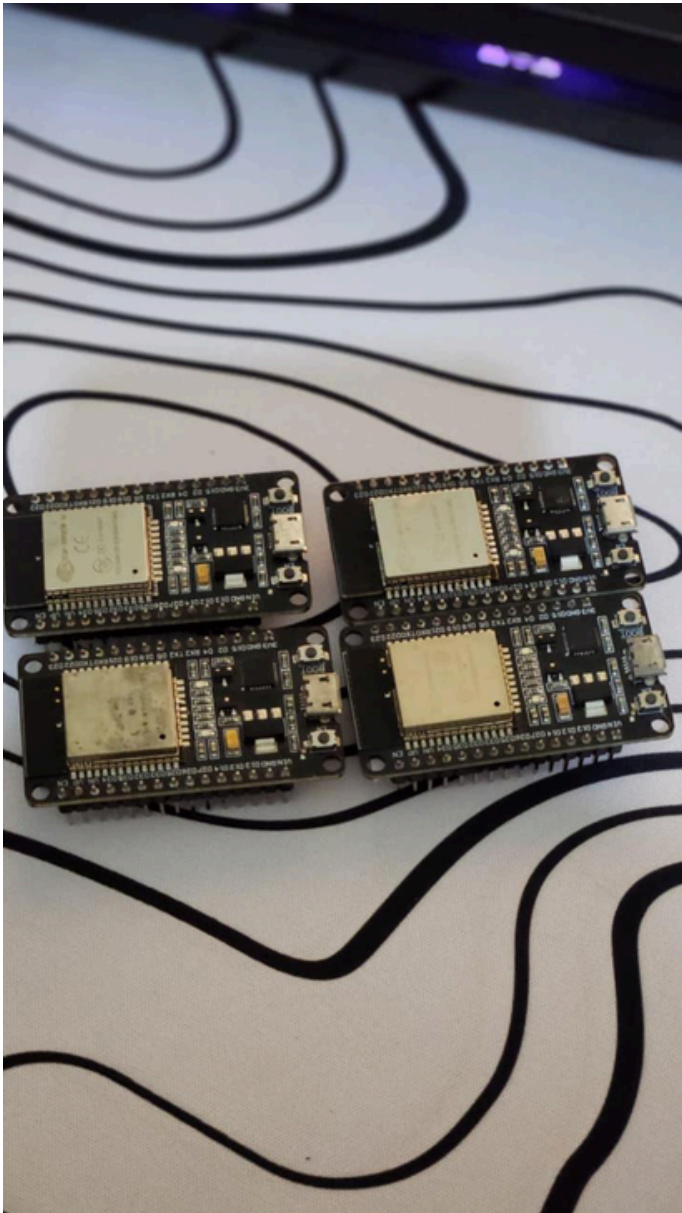
Cada placa foi individualmente submetida a um protocolo de ensaio composto por três etapas consecutivas:

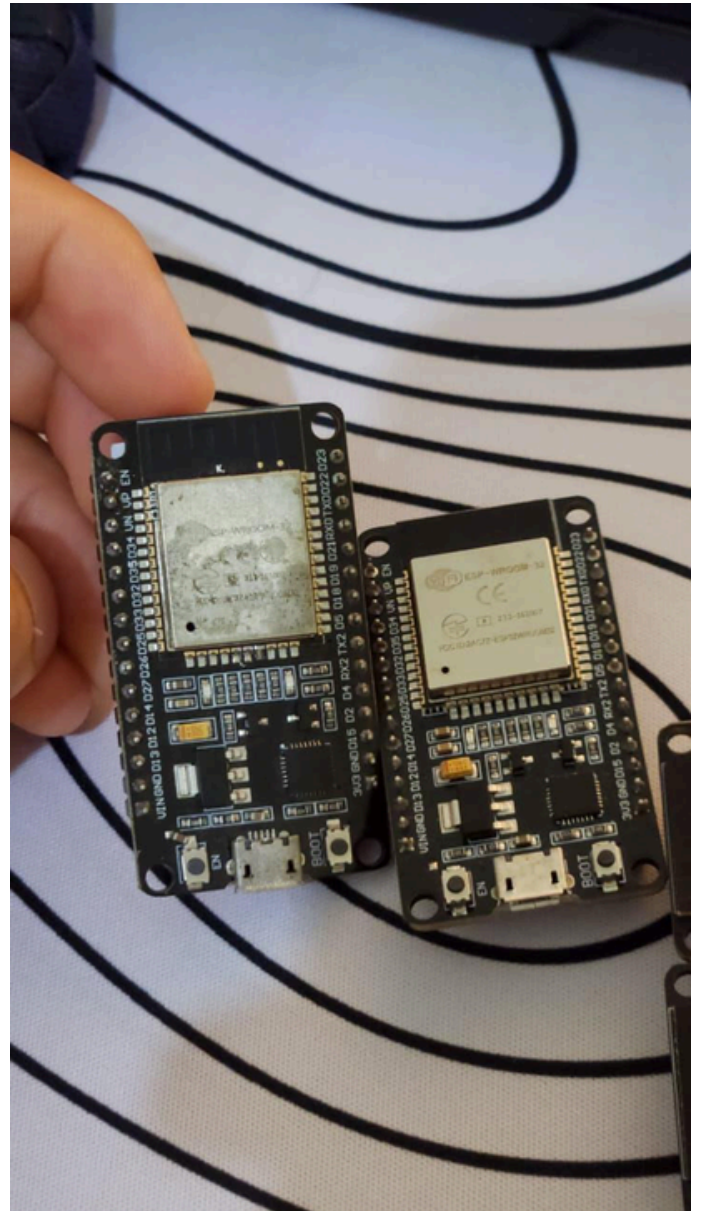
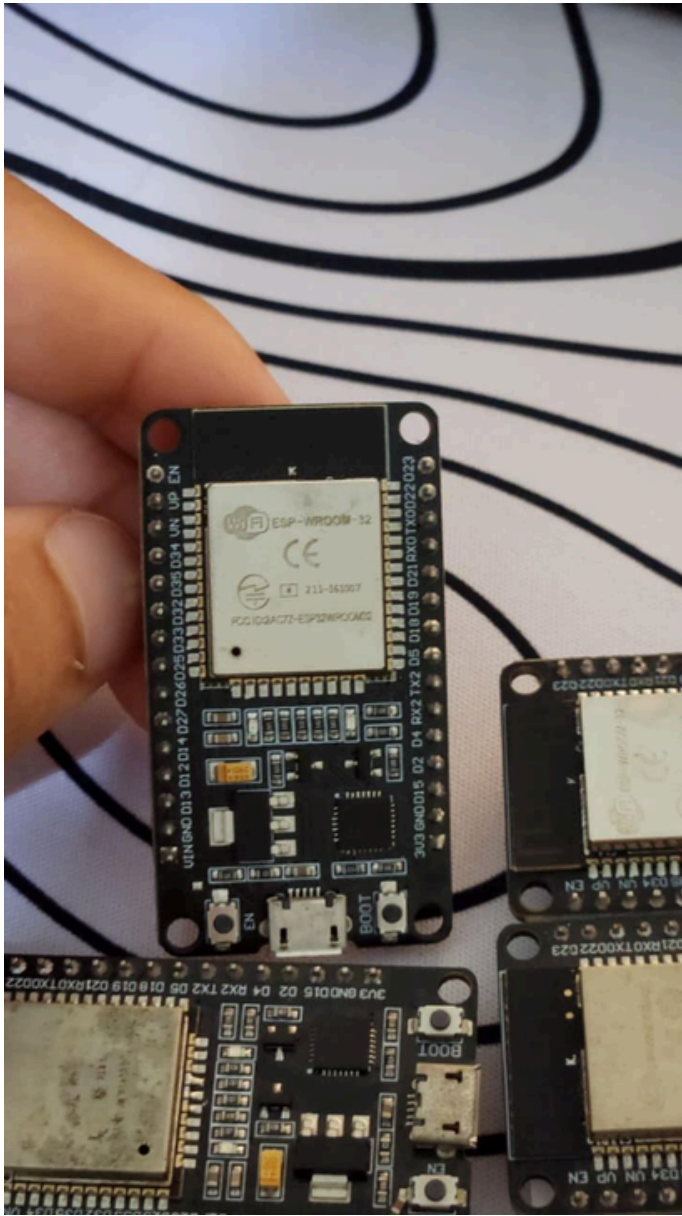
- **Varredura Serial:** Conexão ao computador via cabo USB para testar o reconhecimento da porta COM e a resposta do circuito integrado conversor USB-Serial através do utilitário esptool.
- **Estabilidade de Rede:** Inicialização e permanência de conexão ativa na rede Wi-Fi local por um período mínimo de 10 minutos para avaliar o aquecimento do chip e quedas de sinal.

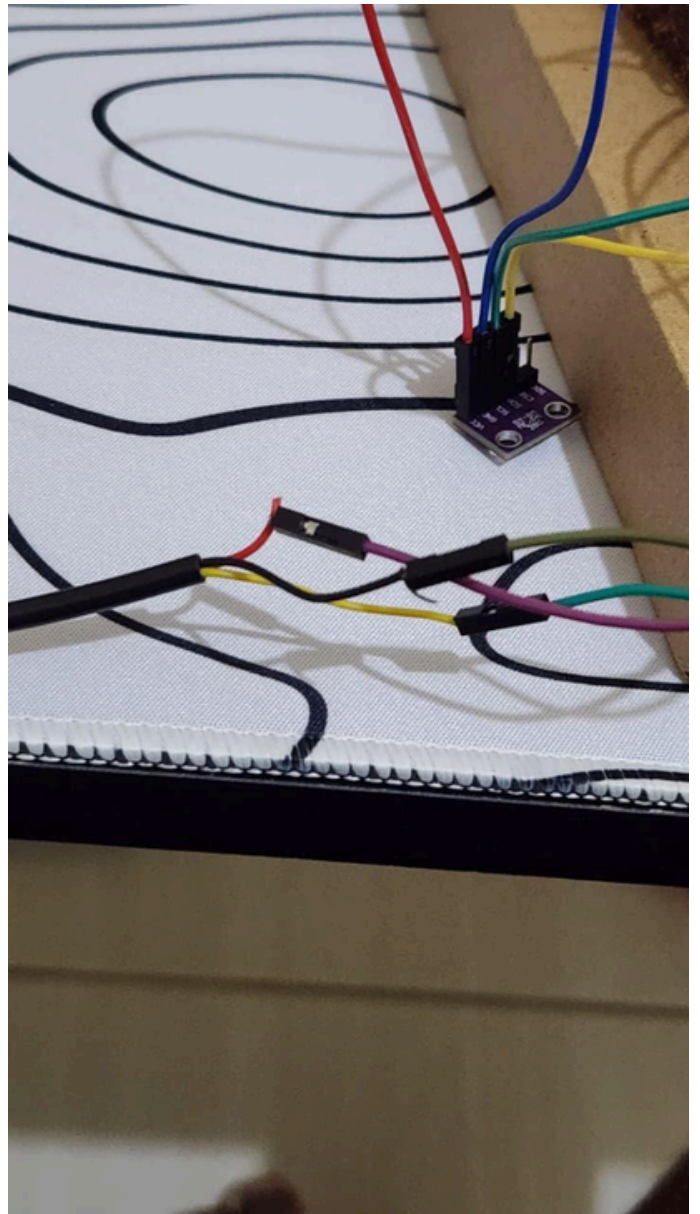
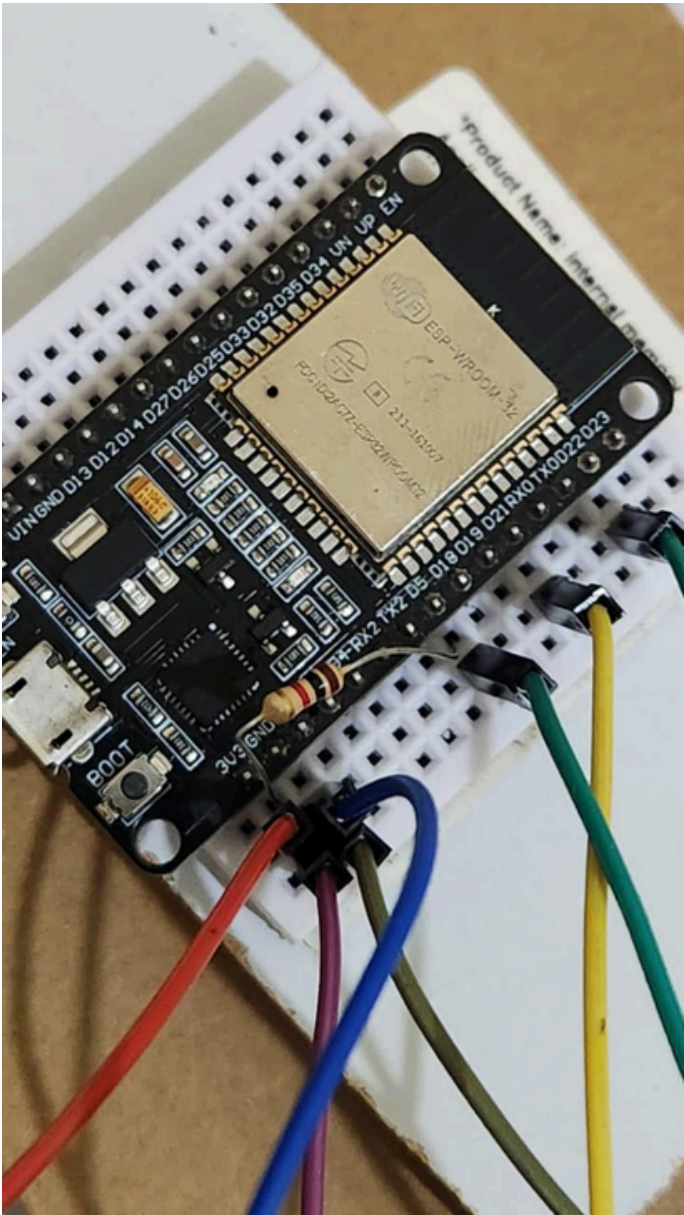
- **Leitura de Periféricos:** Acoplamento aos barramentos I2C e 1-Wire na protoboard para verificar o funcionamento e a estabilidade dos pinos digitais de dados.

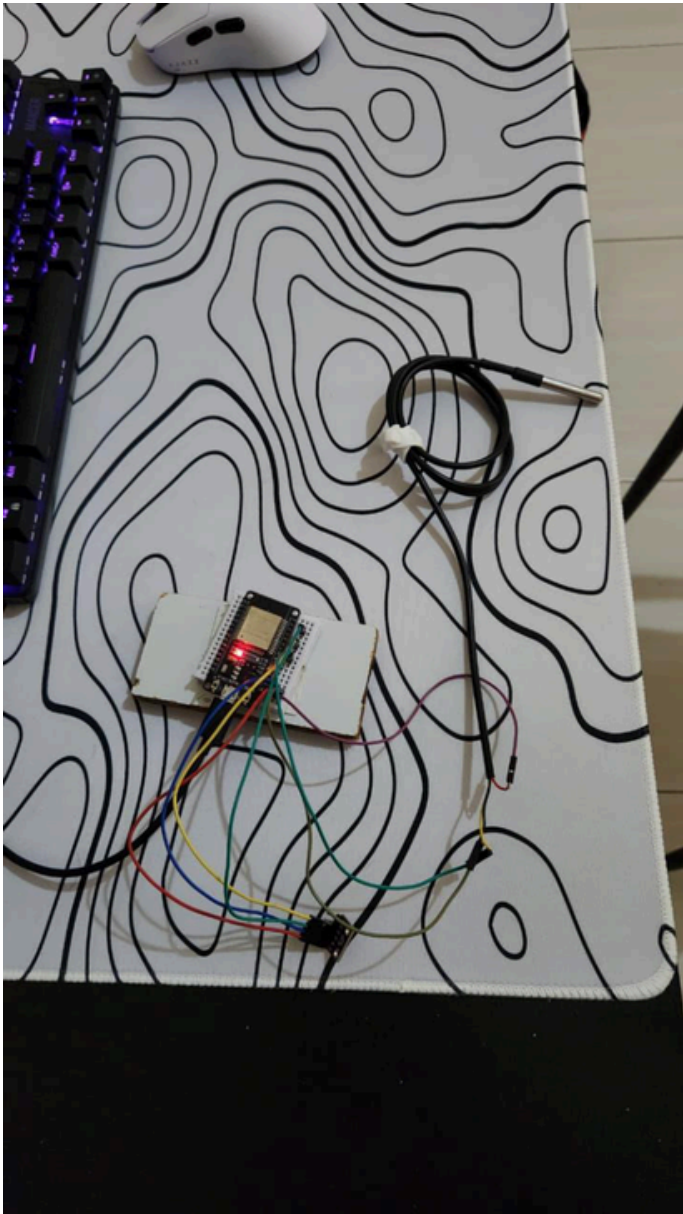
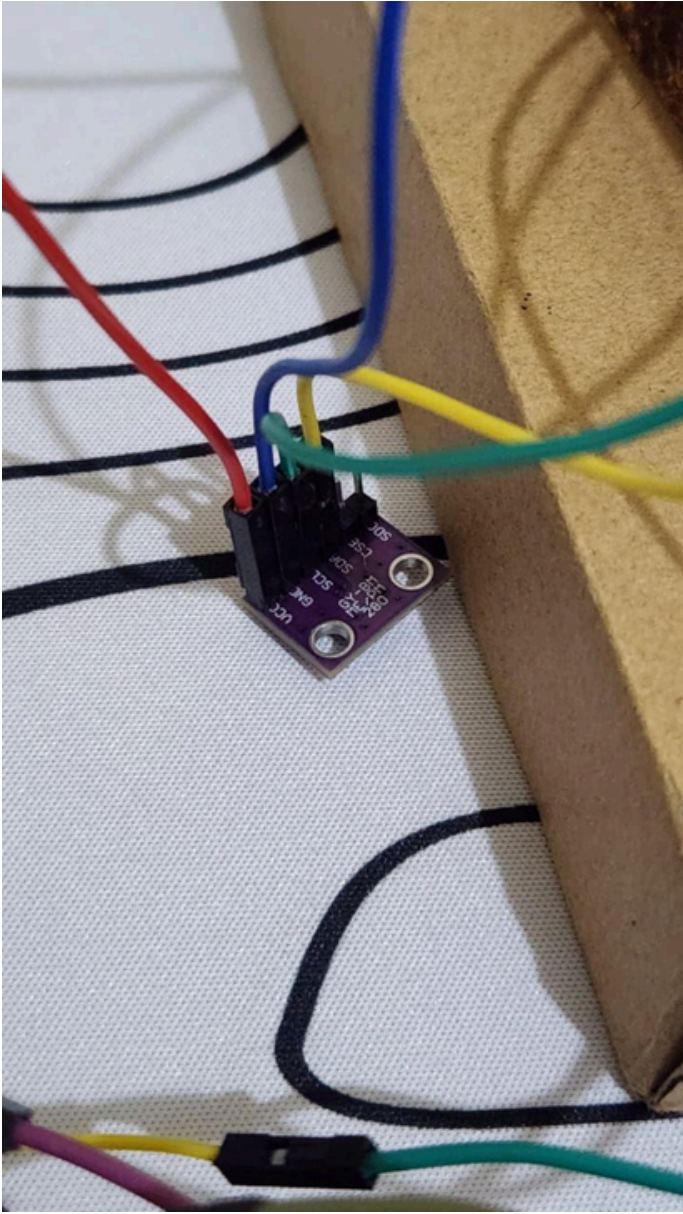
2.2. Mapeamento do Estado das Placas

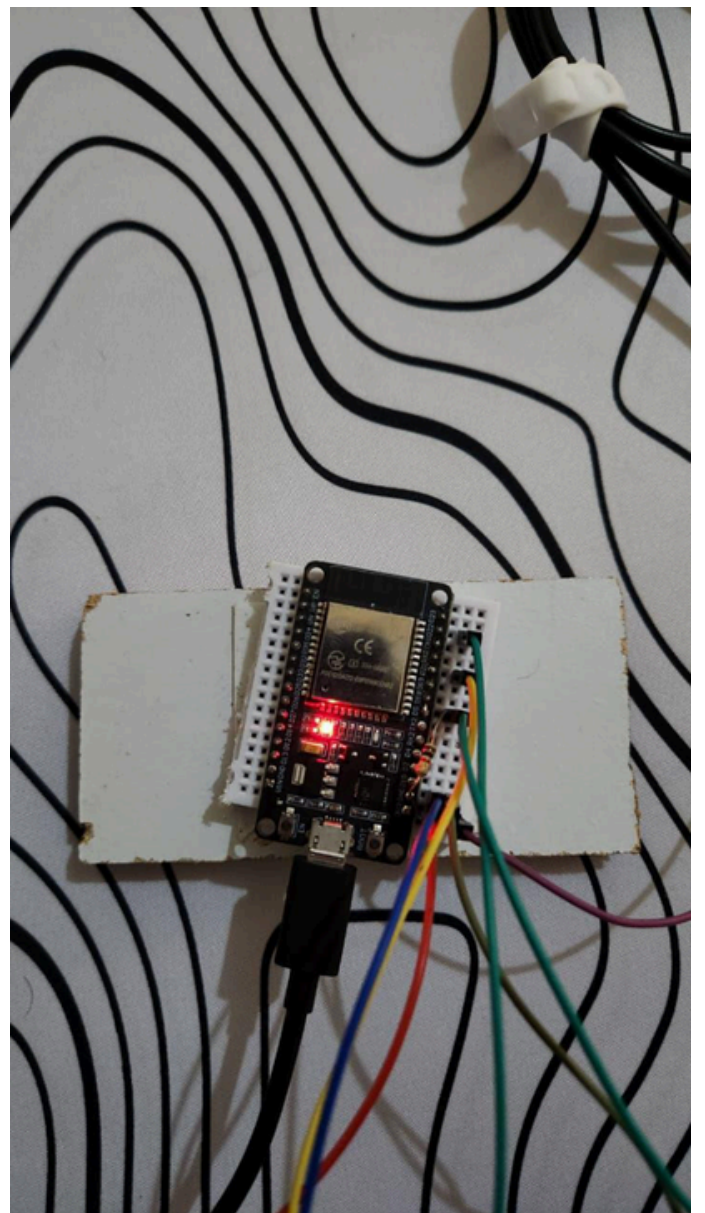
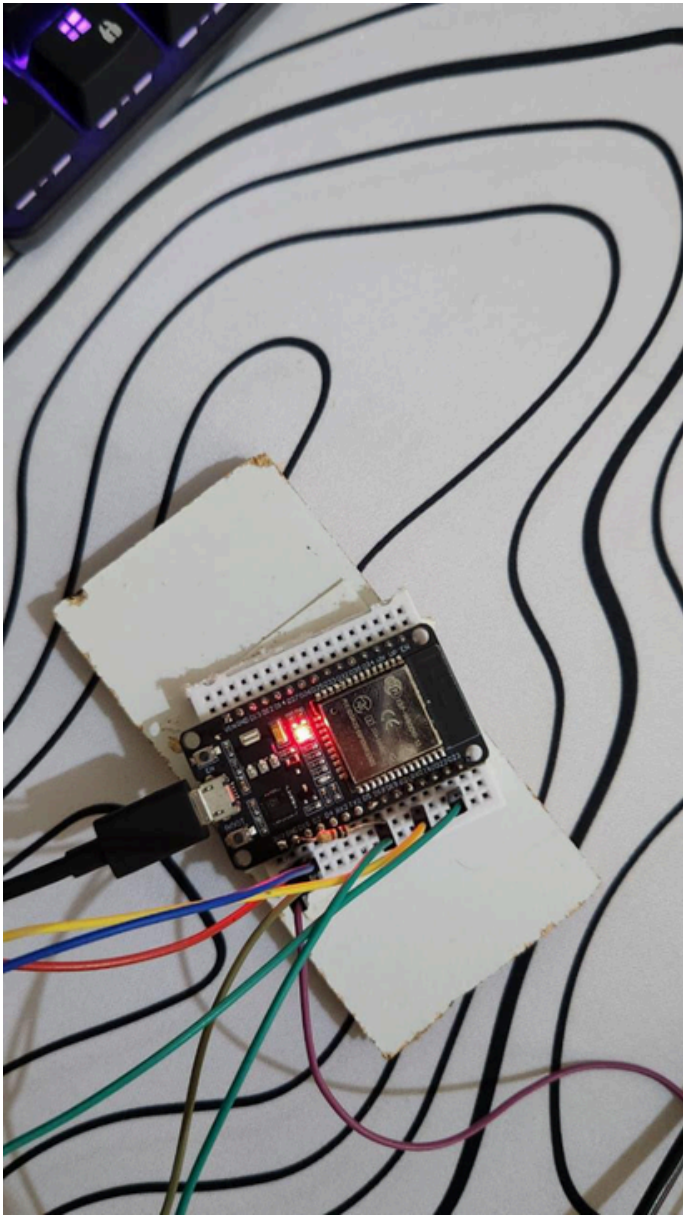
Identificação da Placa	Estado Operacional	Observações Técnicas/ Diagnóstico
ESP32 - Módulo 01 (Principal)	100% Funcional	Placa utilizada no protótipo definitivo. Excelente estabilidade de Wi-Fi e barramentos ativos. Retida para o projeto.
ESP32 - Módulo 02	100% Funcional	Apresentou leituras consistentes em ambos os sensores e obteve resposta estável HTTP 200 (OK) nas requisições enviadas à API Spring Boot. Totalmente operacional, sem perda de pacotes.
ESP32 - Módulo 03	Inoperante	Falha na gravação via esptool na porta COM3. O módulo não estabeleceu comunicação (erro No serial data received). Falha no chip conversor USB-Serial. Isolada para devolução.
ESP32 - Módulo 04		Comportamento idêntico ao Módulo 03. Travamento persistente no loop de conexões (Connecting...), evidenciando quebra do canal serial. Isolada para devolução.

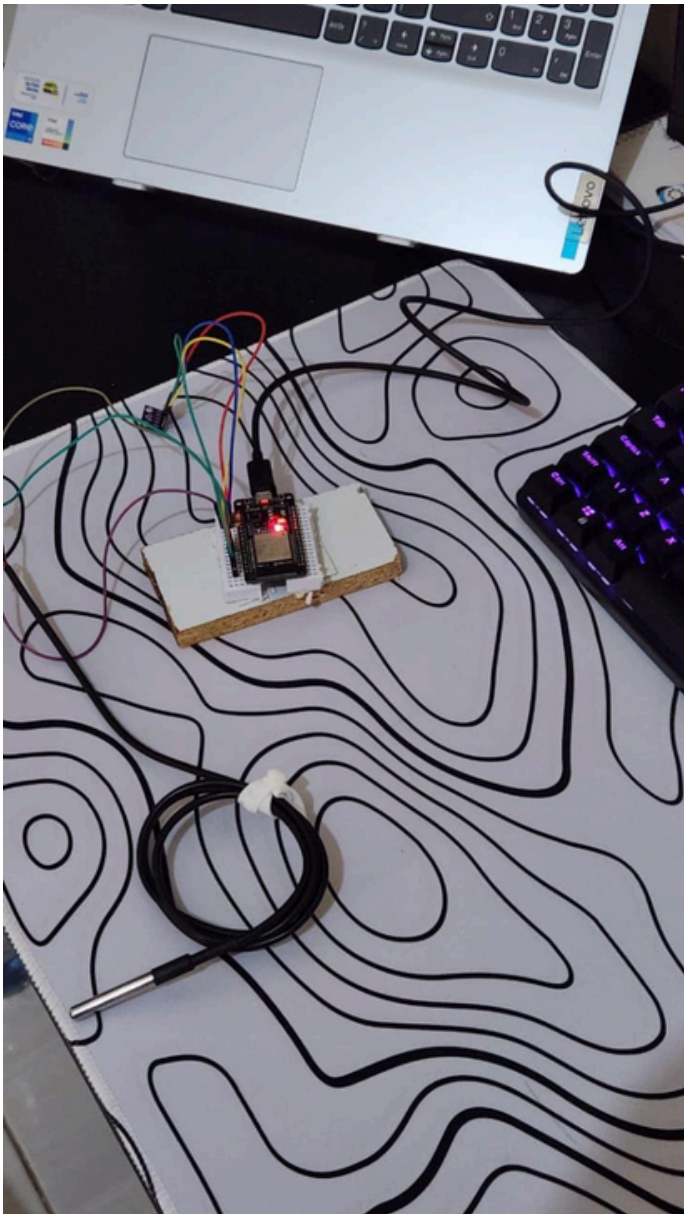
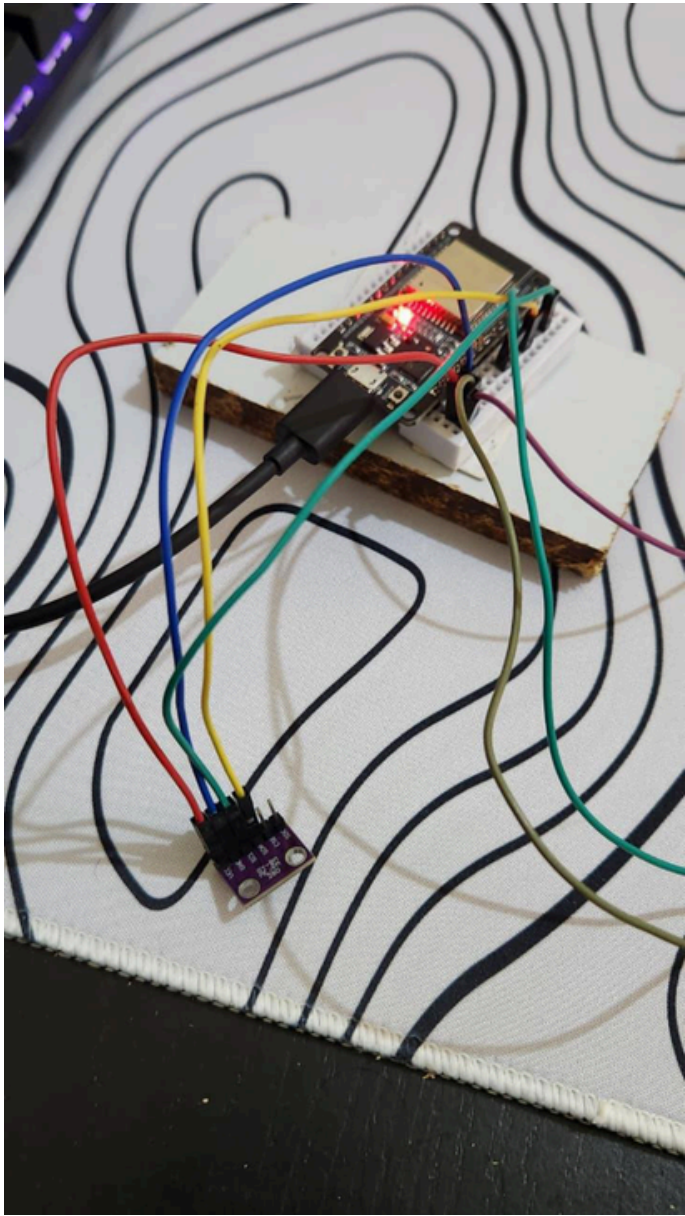












3. CAMADA FÍSICA E DESENVOLVIMENTO DO FIRMWARE (C++)

O código gravado no ESP32 foi desenvolvido em C++ através da Arduino IDE.

A função

principal desse programa é ler os dados físicos dos sensores, empacotar essas

informações em um formato universal e enviá-las para o servidor Java através da

rede Wi-Fi. Para que tudo funcione sem travamentos, o firmware lida com três

pilares: a configuração dos pinos, a montagem do texto de comunicação e o envio dos

pacotes.

3.1. Inicialização dos Sensores e Bibliotecas

No início do código, utilizamos bibliotecas prontas para gerenciar a

comunicação

elétrica com os componentes.

- **Sensor BMP280:** Responsável pela leitura de temperatura e pressão

atmosférica.

É iniciado através da biblioteca da Adafruit, utilizando o barramento síncrono I2C, mapeado nos pinos físicos D21 (SDA) e D22 (SCL) do microcontrolador. O

programa faz uma checagem no início (`begin()`) para garantir que o sensor está alimentado e respondendo.

- **Sensor DS18B20:** Sonda térmica de alta precisão baseada no protocolo 1-Wire, que usa as bibliotecas `OneWire` e `DallasTemperature` no pino D5. Como esse sensor usa apenas um fio para enviar dados, o código precisa disparar um comando de leitura (`requestTemperatures()`) antes de conseguir capturar o valor real da temperatura.

3.2. Montagem do Contrato de Comunicação (JSON)

Como o servidor Java não entende comandos diretos de sensores, o ESP32 traduz os números lidos para o formato JSON. O código foi programado para criar manualmente uma linha de texto organizada com chaves e aspas, separando o nome do sensor de seus respectivos valores.

- Para o BMP280, o ESP32 lê as variáveis de temperatura e pressão e monta o texto exatamente assim:

```
{"temperatura": 30.8, "pressao": 1013.8, "sensor": "BMP280"}
```

- Para o DS18B20, como ele não mede pressão, o código envia o valor fixo de 0.0 apenas para manter a mesma estrutura de dados aceita pelo banco, focando no envio da temperatura:

```
{"temperatura": 29.6, "pressao": 0.0, "sensor": "DS18B20"}
```

3.3. Envio dos Dados via Protocolo HTTP

Com o texto JSON montado, a função de envio entra em ação utilizando a placa de Wi-Fi integrada do ESP32. O programa abre uma conexão direta com o endereço de IP do computador onde o backend em Java está rodando, apontando para a porta 8081 no caminho `/api/sensor/enviar`.

O envio é feito através de uma requisição do tipo HTTP POST (usado para enviar novas informações). No cabeçalho dessa mensagem, o código avisa o servidor que o conteúdo é um `application/json`. Se o Java receber e salvar o dado corretamente, ele responde para o ESP32 com o código 200, que o microcontrolador imprime no Monitor Serial para confirmar que a viagem do dado deu certo.


```
Conectado a rede Wi-Fi!  
BMP280 -> Temp: 31.28 *C | Pressao: 1009.90 hPa  
HTTP BMP280: 200  
DS18B20 -> Temp: 30.44 *C  
HTTP DS18B20: 200  
BMP280 -> Temp: 31.33 *C | Pressao: 1009.89 hPa  
HTTP BMP280: 200  
DS18B20 -> Temp: 30.50 *C  
HTTP DS18B20: 200  
BMP280 -> Temp: 31.36 *C | Pressao: 1009.87 hPa  
HTTP BMP280: 200  
DS18B20 -> Temp: 30.50 *C  
HTTP DS18B20: 200  
BMP280 -> Temp: 31.37 *C | Pressao: 1009.89 hPa  
HTTP BMP280: 200  
DS18B20 -> Temp: 30.50 *C  
HTTP DS18B20: 200
```

3.4. Lógica de Tempo para Evitar Concorrência

Como os sensores enviam dados para a mesma rota do servidor, o código do loop() não despacha as duas leituras exatamente no mesmo milissegundo. Foi programado um intervalo inteligente de 500 milissegundos (meio segundo) entre o envio do BMP280 e o do DS18B20. Isso dá tempo para a rede respirar e impede que um pacote

atrole o outro ao chegar no Java. Depois que os dois são enviados, o circuito aguarda 5 segundos para iniciar a próxima leitura global.

4. ARQUITETURA E INTEGRAÇÃO NO VS CODE (Java / Spring Boot)

O backend do sistema foi desenvolvido em Java utilizando o framework Spring Boot dentro do ambiente do VS Code. O papel dessa camada é funcionar como um servidor que fica escutando a rede na porta 8081, esperando os dados do ESP32 chegarem para guardá-los de forma organizada em um banco de dados local.

Para que essa integração aconteça de forma limpa e sem bagunça no código, o projeto foi dividido em três partes principais, onde cada arquivo tem uma função bem definida:

4.1. A Regra do Banco de Dados (LeituraSensor.java)

Este arquivo é o Modelo do projeto. Ele funciona como uma fôrma que diz ao banco de dados exatamente como as informações devem ser guardadas. Nele, definimos que cada registro de telemetria precisa ter: um número de identificação automático (ID), o valor da temperatura, o valor da pressão (como números decimais) e o nome do sensor que gerou aquela leitura (como texto).

Uma decisão importante de projeto, foi configurar o atributo de data e hora (dataHora) para pegar o horário do sistema, no exato momento em que o dado é

criado no servidor (`LocalDateTime.now()`). Isso garante que o carimbo de tempo seja confiável, já que o relógio interno do computador é muito mais preciso do que o do microcontrolador ESP32.

4.2. O Mecanismo de Salvamento (`SensorRepository.java`)

Este arquivo é o Repositório. Usamos o recurso `JpaRepository` do Spring Boot, que funciona como um intermediário automático: a interface foi criada e o framework nos dá todos os comandos prontos para salvar, listar ou deletar dados. Quando o controlador precisa salvar algo, ele só chama uma função simples (como `.save(dados)`), e o Spring Boot cuida de todo o trabalho pesado de gravação no arquivo do banco de dados H2.

4.3. A Recepção das Requisições (`SensorController.java`)

Este arquivo é o Controlador e serve como a porta de entrada da API. Ele fica vigiando o endereço `/api/sensor` e gerencia as rotas do sistema:

- **Rota de Envio (`/enviar`):** É o endpoint do tipo POST onde o ESP32 bate a cada 5 segundos. Quando o texto JSON do hardware chega aqui, o controlador converte o texto automaticamente em um objeto Java, persiste no banco através do repositório e imprime uma mensagem de sucesso no console do VS Code.
- **Rota de Dados (`/dados`):** É um endpoint do tipo GET. Quando o Dashboard chama essa rota, o controlador vai até o banco, pega o histórico completo de leituras e devolve em formato JSON para alimentar o gráfico.
- **Controle de Acesso (`@CrossOrigin`):** Foi adicionada essa configuração no topo do controlador para resolver o problema de segurança de acesso cruzado (erro de CORS). Sem essa linha, os navegadores modernos bloqueariam a página HTML de puxar os dados do gráfico diretamente do servidor Java.

5. CAMADA DE VISÃO E VALIDAÇÃO DOS DADOS (`Dashboard`)

O painel web foi integrado na diretoria de recursos estáticos do servidor Spring Boot (`src/main/resources/static/dashboard.html`). Isso permite que ele seja acessado no navegador digitando apenas `http://localhost:8081/dashboard.html`. A construção do painel foi dividida em três frentes principais: o visual da tela, a captura dos dados e a lógica de separação do gráfico.

5.1. Implementação Técnica do Dashboard

- 1. Interface Visual (HTML e Bootstrap):** Foi utilizado o Bootstrap, que é uma biblioteca de CSS pronta para o design. Com ele, a página foi estruturada com três blocos superiores (cards) que mostram o último valor de temperatura e pressão em tempo real. As cores das bordas foram configuradas para mudar (azul para o BMP280 e amarelo para o DS18B20) para facilitar a leitura visual rápida. Logo abaixo, foi reservado um espaço para o gráfico principal ocupar a tela inteira.
- 2. Consumo da API em Tempo Real (JavaScript e Fetch):** Para fazer a página atualizar sozinha, sem que o usuário precise apertar F5, foi criada uma função em JavaScript que usa o comando `fetch()`. Esse comando faz um pedido para a rota do servidor Java (`/api/sensor/dados`), puxa o histórico completo que está guardado no banco e o transforma em uma lista de objetos na memória do navegador. Para manter o monitoramento contínuo, foi configurado um temporizador (`setInterval`) que repete essa busca automaticamente a cada 3 segundos.
- 3. O Desafio do Gráfico e a Solução do Filtro (Chart.js):** Para gerar o gráfico de linhas, foi utilizada a biblioteca `Chart.js`. O maior desafio técnico dessa etapa surgiu por conta da estrutura do banco de dados: como a API salva as leituras do BMP280 e do DS18B20 de forma intercalada na mesma tabela, os dados chegam para o navegador em uma fila única e misturada. Se essa lista fosse passada direto para o gráfico, a linha tentaria ligar os pontos sequencialmente, gerando um efeito de 'zigue-zague' confuso que cruzaria as temperaturas dos dois sensores. Para resolver esse problema, o método `.filter()` do JavaScript foi aplicado para funcionar como um separador de arquivos na memória do navegador:

```
const dadosBMP = todosOsDados.filter(item=>
item.sensor === 'BMP280');
const dadosDS = todosOsDados.filter(item=>
item.sensor === 'DS18B20');
```

Uma linha filtra apenas o que pertence ao BMP280 e joga em uma lista isolada, enquanto a outra linha faz o mesmo filtrando apenas o DS18B20. Com as duas listas devidamente separadas e organizadas, o `Chart.js` conseguiu plotar duas linhas independentes e paralelas no mesmo gráfico. Isso garantiu uma visualização limpa para comparar o comportamento térmico dos dois sensores ao longo do tempo.

Painel de Monitoramento Híbrido (IoT)

Leituras simultâneas dos sensores BMP280 e DS18B20 integradas ao Backend Java

BMP280 - TEMPERATURA

31.7 °C

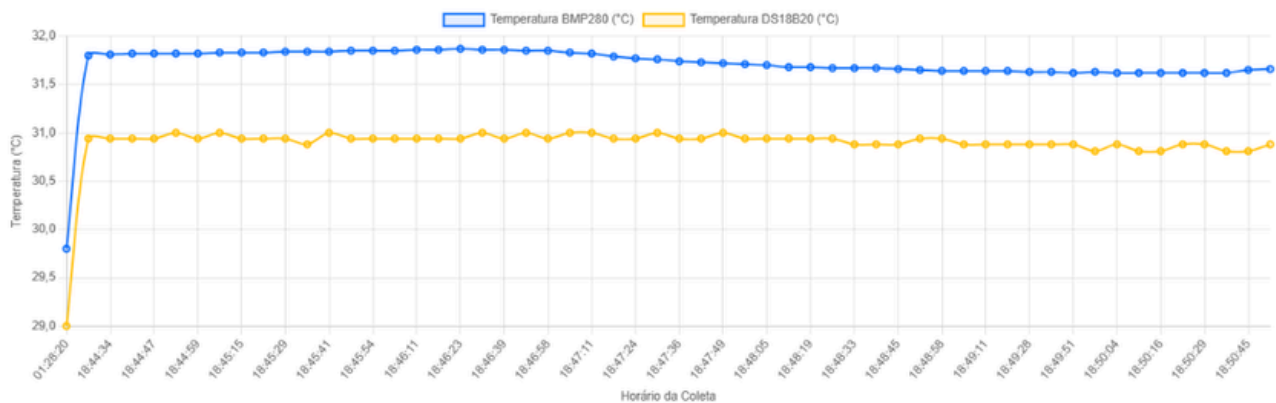
BMP280 - PRESSÃO

1011.3 hPa

DS18B20 - TEMPERATURA

30.8 °C

Gráfico Comparativo de Estabilidade Térmica



5.2. Registros de Logs do Servidor

O trecho de log obtido diretamente no terminal do VS Code durante o ensaio com o Módulo 02 comprova o sucesso da gravação sequencial e o recebimento das informações vindas do microcontrolador:

```
Salvo no banco com ID: 3049 | Sensor: BMP280
Salvo no banco com ID: 3050 | Sensor: DS18B20
Salvo no banco com ID: 3051 | Sensor: BMP280
Salvo no banco com ID: 3052 | Sensor: DS18B20
Salvo no banco com ID: 3053 | Sensor: BMP280
Salvo no banco com ID: 3054 | Sensor: DS18B20
Salvo no banco com ID: 3055 | Sensor: BMP280
Salvo no banco com ID: 3056 | Sensor: DS18B20
Salvo no banco com ID: 3057 | Sensor: BMP280
Salvo no banco com ID: 3058 | Sensor: DS18B20
Salvo no banco com ID: 3059 | Sensor: BMP280
Salvo no banco com ID: 3060 | Sensor: DS18B20
Salvo no banco com ID: 3061 | Sensor: BMP280
Salvo no banco com ID: 3062 | Sensor: DS18B20
Salvo no banco com ID: 3063 | Sensor: BMP280
Salvo no banco com ID: 3064 | Sensor: DS18B20
Salvo no banco com ID: 3065 | Sensor: BMP280
Salvo no banco com ID: 3066 | Sensor: DS18B20
Salvo no banco com ID: 3067 | Sensor: BMP280
Salvo no banco com ID: 3068 | Sensor: DS18B20
Salvo no banco com ID: 3069 | Sensor: BMP280
Salvo no banco com ID: 3070 | Sensor: DS18B20
```

5.3. Análise Prática dos Logs

A análise das mensagens geradas no console do servidor permite validar três pontos importantes sobre o funcionamento do ecossistema:

- **Garantia de Não-Nulidade:** A presença explícita das strings 'BMP280' e 'DS18B20' no parâmetro Sensor: valida o correto funcionamento do motor de conversão do Java. Isso atesta que o formato enviado pelo ESP32 está perfeitamente alinhado com o servidor, e que nenhuma informação foi gravada como vazia (null).
- **Consistência Sequencial:** A geração estritamente sequencial das chaves primárias (ID: 3049 a 3070) controlada pelo banco de dados local H2, comprova que o armazenamento em arquivo está operando de forma rápida, sem travamentos de escrita ou perda de registros.
- **Sincronismo de Coleta Híbrida:** O comportamento rigorosamente intercalado (um registro de cada componente por vez) valida na prática a eficácia da lógica de tempo configurada no firmware do ESP32, garantindo que os pacotes viajem sem atropelar a recepção do servidor backend

5.4. Volumetria e Distribuição dos Testes de Estresse

Para validar a resiliência do ecossistema e quantificar o volume de dados trafegados durante os ensaios de bancada, o histórico do banco de dados H2 foi auditado com base nos intervalos de chaves primárias (IDs) gerados sequencialmente.

Como a arquitetura foi desenhada para que cada ciclo completo de coleta execute duas requisições HTTP POST consecutivas e intercaladas (o ID ímpar correspondendo ao BMP280 e o ID par ao DS18B20), o volume geral de dados dividiu-se nos seguintes intervalos controlados (No que se refere aos testes finais):

- **Ensaio do Módulo 01 (Placa Principal):** Operou de forma contínua entre os IDs **1865 e 2481**, resultando em 616 registros individuais salvos com sucesso (o que equivale a 308 ciclos completos de telemetria híbrida).
- **Ensaio do Módulo 02 (Placa Homologada):** Operou no intervalo final entre os IDs **2481 e 3019**, gerando 538 registros individuais salvos com sucesso (o que equivale a 269 ciclos completos de telemetria híbrida).

Conclusão da Amostragem: O banco de dados ultrapassou a marca de 3000 entradas individuais armazenadas. Desse total, foram processados e validados aproximadamente **1500 ciclos completos de telemetria** entre hardware e backend.

6. PRINCIPAIS DIFICULDADES E COMO FORAM RESOLVIDAS

Durante o desenvolvimento do projeto, surgiram alguns problemas comuns de integração entre hardware e software. A resolução de cada um deles serviu como aprendizado prático de como alinhar o código do microcontrolador com o do servidor.

1. O Problema do Sensor Salvando como Vazio (Sensor: null)

O que aconteceu: Nos primeiros testes, o ESP32 enviava os dados, o Java recebia, mas na hora de salvar no banco de dados, o campo que deveria mostrar o nome do sensor (BMP280 ou DS18B20) ficava escrito null (vazio).

Como foi resolvido: O erro estava na forma como a string de texto era montada na Arduino IDE. O Java é muito exigente com os nomes das variáveis. Para corrigir, o código em C++ foi ajustado usando barras invertidas para "escapar" as aspas (\"sensor\"). Isso conseguiu garantir que o texto enviado pelo ESP32 ficasse exatamente igual ao nome do atributo esperado pelo Java.

2. Travamento e Conflito de Tabelas no Banco de Dados (H2)

O que aconteceu: Sempre que era preciso reiniciar o servidor Java no VS Code para testar alguma alteração, o banco de dados H2 entrava em conflito com as informações antigas e dava erro na hora de atualizar a tabela das leituras.

Como foi resolvido: O comando de atualização automática do Spring Boot estava se confundindo com os arquivos temporários guardados no computador. A solução foi ir até a pasta do projeto, apagar manualmente a pastinha de dados locais (./data/telemetria) e ajustar as configurações do arquivo application.properties para forçar o banco a recriar a tabela do zero de forma limpa.

3. Valores Congelados e Mau Contato na Protoboard

O que aconteceu: Durante alguns testes práticos na bancada, o gráfico de temperatura do BMP280 de repente virava uma linha reta horizontal (congelava em uma temperatura fixa) e não mudava mesmo se o sensor fosse aquecido manualmente.

Como foi resolvido: Esse não era um erro de programação, mas sim um problema físico do hardware. Como o barramento de comunicação física (I2C) é muito sensível a pequenas vibrações, quando mexia na protoboard, algum jumper dava mau contato e travava o chip do sensor. O problema foi resolvido revisando o aperto de todos os fios na placa e reiniciando o circuito elétrico pelo cabo USB.

7.CONCLUSÃO

O sistema híbrido desenvolvido cumpre integralmente os requisitos operacionais estabelecidos pelo corpo docente. O ecossistema encontra-se em regime estável de

funcionamento, gerando curvas térmicas consistentes e fornecendo ferramentas ágeis de exportação de dados (CSV) para fins de validação científica e continuidade do

projeto de pesquisa.